DATA CLEANING PROCESS

NAME: JEYA HARSHINI R

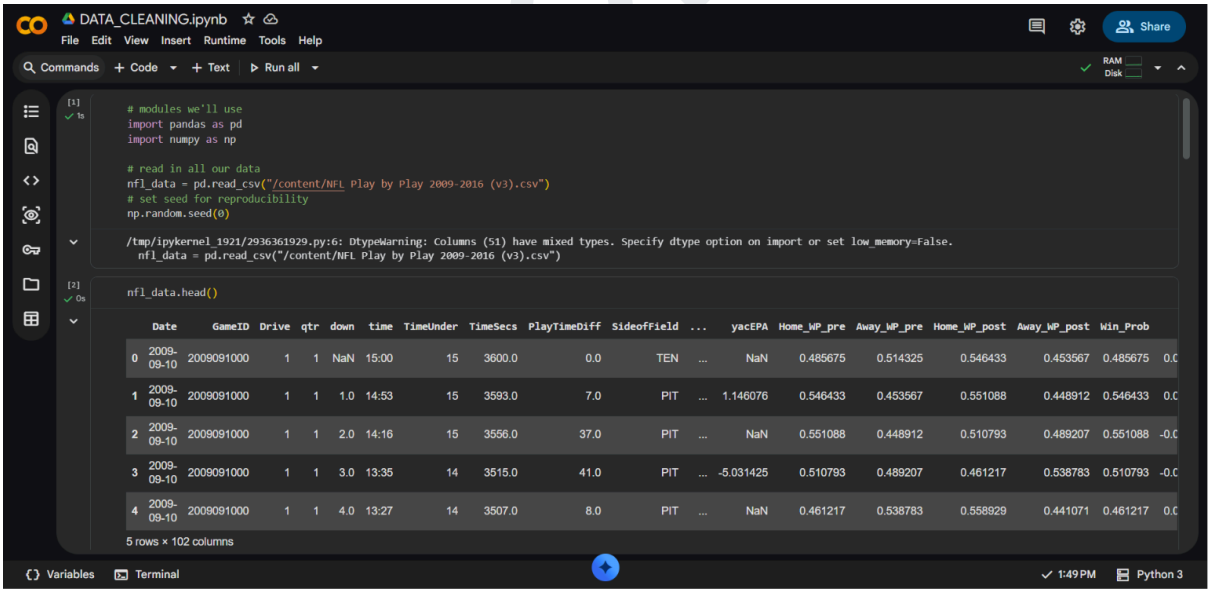
REG NO: 192424051

Step: Load the Dataset (Raw Data)

Description:
The required Python libraries, **Pandas** and **NumPy**, are imported to perform data analysis and cleaning operations. The NFL Play-by-Play dataset is then loaded into a Pandas DataFrame using the `read_csv()` function.

Purpose:

- Import data manipulation libraries.
- Load the raw dataset into memory.
- Prepare the dataset for cleaning and analysis.



Step: Understanding the Dataset

Description:
The `head()` function displays the first five rows of the dataset, allowing us to inspect the structure, column names, data types, and sample values before performing any cleaning operations.

Purpose:

- Understand dataset structure.
- Verify successful data loading.
- Identify important features.

```
[2]
nfl_data.head()
```

	Date	GameID	Drive	qtr	down	time	TimeUnder	TimeSecs	PlayTimeDiff	SideofField	...	yacEPA	Home_WP_pre	Away_WP_pre	Home_WP_post	Away_WP_post	Win_Prob	
0	2009-09-10	2009091000	1	1	NaN	15:00	15	3600.0	0.0	TEN	...	NaN	0.485675	0.514325	0.546433	0.453567	0.485675	0.0
1	2009-09-10	2009091000	1	1	1.0	14:53	15	3593.0	7.0	PIT	...	1.146076	0.546433	0.453567	0.551088	0.448912	0.546433	0.0
2	2009-09-10	2009091000	1	1	2.0	14:16	15	3556.0	37.0	PIT	...	NaN	0.551088	0.448912	0.510793	0.489207	0.551088	-0.0
3	2009-09-10	2009091000	1	1	3.0	13:35	14	3515.0	41.0	PIT	...	-5.031425	0.510793	0.489207	0.461217	0.538783	0.510793	-0.0
4	2009-09-10	2009091000	1	1	4.0	13:27	14	3507.0	8.0	PIT	...	NaN	0.461217	0.538783	0.558929	0.441071	0.461217	0.0

5 rows × 102 columns

Step: Detect Missing Values

Description:
The `isnull().sum()` function counts the number of missing (NaN) values present in each column. This helps identify which attributes require cleaning.

Purpose:

- Locate missing values.
- Measure missing data per column.
- Decide appropriate cleaning techniques.

```
[3]
# get the number of missing data points per column
missing_values_count = nfl_data.isnull().sum()

# look at the # of missing points in the first ten columns
missing_values_count[0:10]
```

	0
Date	0
GameID	0
Drive	0
qtr	0
down	8743
time	40
TimeUnder	0
TimeSecs	40
PlayTimeDiff	80
SideofField	82

dtype: int64

Step: Calculate Missing Data Percentage

Description:
The total number of missing values is calculated and divided by the total number of cells in the dataset to determine the percentage of missing data.

Result:

- Missing Data = **27.75%**

Purpose:

- Measure dataset quality.
- Evaluate how much cleaning is required.

```
[5]
# how many total missing values do we have?
total_cells = np.prod(nfl_data.shape)
total_missing = missing_values_count.sum()

# percent of data that is missing
percent_missing = (total_missing/total_cells) * 100
print(percent_missing)

27.75171178232892

[6]
# look at the # of missing points in the first ten columns
missing_values_count[0:10]
```

	0
Date	0
GameID	0
Drive	0
qtr	0
down	8743
time	40
TimeUnder	0
TimeSecs	40
PlayTimeDiff	80

Step: Verify Missing Value Count

Description:
The missing value count is displayed again to verify the number of missing entries before applying cleaning techniques.

Purpose:

- Confirm missing value statistics.
- Ensure accurate preprocessing.

```
[7]
# remove all the rows that contain a missing value
nfl_data.dropna()

Date GameID Drive qtr down time TimeUnder TimeSecs PlayTimeDiff SideofField ... yacEPA Home_WP_pre Away_WP_pre Home_WP_post Away_WP_post Win_Prob WPA airWPA
0 rows × 102 columns
```

Step: Drop Rows Containing Missing Values

Description:
The `dropna()` function removes every row that contains at least one missing value.

Observation:
The result shows **0 rows remaining**, indicating that every row contained at least one missing value.

Purpose:

- Remove incomplete records.
- Demonstrate the effect of row deletion.

```
[8]
# remove all columns with at least one missing value
columns_with_na_dropped = nfl_data.dropna(axis=1)
columns_with_na_dropped.head()
```

	Date	GameID	Drive	qtr	TimeUnder	ydstogo	ydsnet	desc	PlayAttempted	Yards.Gained	...	HomeTeam	AwayTeam	Timeout_Indicator	posteam_timeouts_pre	Home
0	2009-09-10	2009091000	1	1	15	0	0	R.Bironas kicks 67 yards from TEN 30 to PIT 3....	1	39	...	PIT	TEN	0	3	
1	2009-09-10	2009091000	1	1	15	10	5	(14:53) B.Roethlisberger pass short left to H....	1	5	...	PIT	TEN	0	3	
2	2009-09-10	2009091000	1	1	15	5	2	(14:16) W.Parker right end to PIT 44 for -3 ya...	1	-3	...	PIT	TEN	0	3	
3	2009-09-10	2009091000	1	1	14	8	2	(13:35) (Shotgun) B.Roethlisberger pass incomp...	1	0	...	PIT	TEN	0	3	
4	2009-09-10	2009091000	1	1	14	8	2	(13:27) (Punt formation) D.Sepulveda punts 54 ...	1	0	...	PIT	TEN	0	3	

5 rows × 37 columns

Step: Drop Columns Containing Missing Values

Description:
The `dropna(axis=1)` function removes every column containing at least one missing value.

Result:

- Original columns: **102**
- Remaining columns: **37**

Purpose:

- Retain only complete columns.
- Create a dataset without missing attributes.

```
[9]
# just how much data did we lose?
print("Columns in original dataset: %d \n" % nfl_data.shape[1])
print("Columns with na's dropped: %d" % columns_with_na_dropped.shape[1])

Columns in original dataset: 102

Columns with na's dropped: 37
```

Step: Validate Data Cleaning

Description:
The number of columns before and after cleaning is compared to measure the amount of information removed during preprocessing.

Result:

- Original Dataset = **102 columns**
- Clean Dataset = **37 columns**

Purpose:

- Evaluate cleaning impact.
- Validate preprocessing results.

```
[11]
✓ 0s
# replace all NA's with 0
subset_nfl_data.fillna(0)
```

	EPA	airEPA	yacEPA	Home_WP_pre	Away_WP_pre	Home_WP_post	Away_WP_post	Win_Prob	WPA	airWPA	yacWPA	Season
0	2.014474	0.000000	0.000000	0.485675	0.514325	0.546433	0.453567	0.485675	0.060758	0.000000	0.000000	2009.0
1	0.077907	-1.068169	1.146076	0.546433	0.453567	0.551088	0.448912	0.546433	0.004655	-0.032244	0.036899	2009.0
2	-1.402760	0.000000	0.000000	0.551088	0.448912	0.510793	0.489207	0.551088	-0.040295	0.000000	0.000000	2009.0
3	-1.712583	3.318841	-5.031425	0.510793	0.489207	0.461217	0.538783	0.510793	-0.049576	0.106663	-0.156239	2009.0
4	2.097796	0.000000	0.000000	0.461217	0.538783	0.558929	0.441071	0.461217	0.097712	0.000000	0.000000	2009.0

Step: Select Important Features

Description:
A smaller subset of relevant columns is created from the original dataset. This subset will be used to demonstrate different missing value handling techniques.

Purpose:

- Reduce dataset complexity.
- Focus only on selected variables.

```
[10]
✓ 0s
# get a small subset of the NFL dataset
subset_nfl_data = nfl_data.loc[:, 'EPA':'Season'].head()
subset_nfl_data
```

	EPA	airEPA	yacEPA	Home_WP_pre	Away_WP_pre	Home_WP_post	Away_WP_post	Win_Prob	WPA	airWPA	yacWPA	Season
0	2.014474	NaN	NaN	0.485675	0.514325	0.546433	0.453567	0.485675	0.060758	NaN	NaN	2009.0
1	0.077907	-1.068169	1.146076	0.546433	0.453567	0.551088	0.448912	0.546433	0.004655	-0.032244	0.036899	2009.0
2	-1.402760	NaN	NaN	0.551088	0.448912	0.510793	0.489207	0.551088	-0.040295	NaN	NaN	2009.0
3	-1.712583	3.318841	-5.031425	0.510793	0.489207	0.461217	0.538783	0.510793	-0.049576	0.106663	-0.156239	2009.0
4	2.097796	NaN	NaN	0.461217	0.538783	0.558929	0.441071	0.461217	0.097712	NaN	NaN	2009.0

Next steps: Generate code with subset_nfl_data | New interactive sheet

Step: Constant Value Imputation

Description:
The `fillna(0)` function replaces every missing value with **0**.

Purpose:

- Eliminate missing values.
- Maintain dataset size.
- Apply simple imputation.

```
[12]
# replace all NA's the value that comes directly after it in the same column,
# then replace all the remaining na's with 0
subset_nfl_data.fillna(method='bfill', axis=0).fillna(0)
```

```
/tmp/ipykernel_1921/3049679174.py:3: FutureWarning: DataFrame.fillna with 'method' is deprecated and will raise in a future version. Use obj.ffill() or obj.bfill() instead.
  subset_nfl_data.fillna(method='bfill', axis=0).fillna(0)
```

	EPA	airEPA	yacEPA	Home_WP_pre	Away_WP_pre	Home_WP_post	Away_WP_post	Win_Prob	WPA	airWPA	yacWPA	Season
0	2.014474	-1.068169	1.146076	0.485675	0.514325	0.546433	0.453567	0.485675	0.060758	-0.032244	0.036899	2009.0
1	0.077907	-1.068169	1.146076	0.546433	0.453567	0.551088	0.448912	0.546433	0.004655	-0.032244	0.036899	2009.0
2	-1.402760	3.318841	-5.031425	0.551088	0.448912	0.510793	0.489207	0.551088	-0.040295	0.106663	-0.156239	2009.0
3	-1.712583	3.318841	-5.031425	0.510793	0.489207	0.461217	0.538783	0.510793	-0.049576	0.106663	-0.156239	2009.0
4	2.097796	0.000000	0.000000	0.461217	0.538783	0.558929	0.441071	0.461217	0.097712	0.000000	0.000000	2009.0

Step: Import Libraries for Scaling and Normalization

Description:

The required Python libraries are imported to perform scaling, normalization, statistical transformations, and data visualization. The `scipy.stats` module provides the Box-Cox transformation, while `mlxtend.preprocessing` is used for Min-Max Scaling. Seaborn and Matplotlib are used to visualize the results before and after transformation.

Purpose:

- Import preprocessing libraries.
- Enable scaling and normalization techniques.
- Prepare the environment for data visualization.

```
[13]
# modules we'll use
import pandas as pd
import numpy as np

# for Box-Cox Transformation
from scipy import stats

# for min_max scaling
from mlxtend.preprocessing import minmax_scaling

# plotting modules
import seaborn as sns
import matplotlib.pyplot as plt

# set seed for reproducibility
np.random.seed(0)
```

Step: Min-Max Scaling

Description:

Randomly generated exponential data is scaled using the Min-Max Scaling technique. This transformation rescales every value to lie between **0 and 1** while preserving the original distribution of the data.

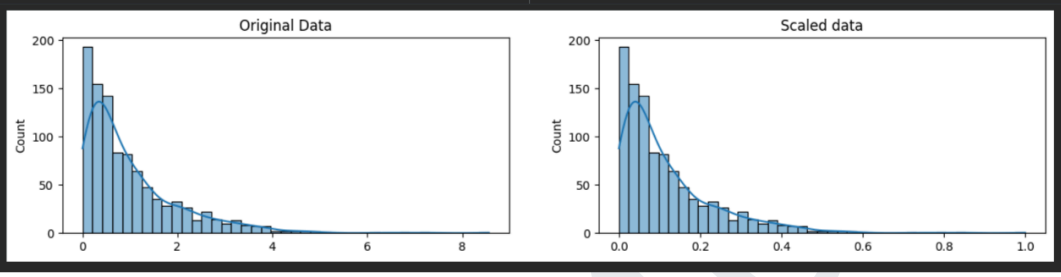
Observation:

The scaled histogram has the same shape as the original distribution, but all values now fall within the range **0–1**.

Purpose:

- Normalize the range of feature values.
- Improve compatibility with machine learning algorithms.
- Prevent features with large values from dominating the analysis.

```python
# generate 1000 data points randomly drawn from an exponential distribution
original_data = np.random.exponential(size=1000)

# mix-max scale the data between 0 and 1
scaled_data = minmax_scaling(original_data, columns=[0])

# plot both together to compare
fig, ax = plt.subplots(1, 2, figsize=(15, 3))
sns.histplot(original_data, ax=ax[0], kde=True, legend=False)
ax[0].set_title("Original Data")
sns.histplot(scaled_data, ax=ax[1], kde=True, legend=False)
ax[1].set_title("Scaled data")
plt.show()
```



Step: Box-Cox Normalization

Description:

The Box-Cox transformation is applied to reduce skewness and make the exponential distribution more closely resemble a normal distribution.

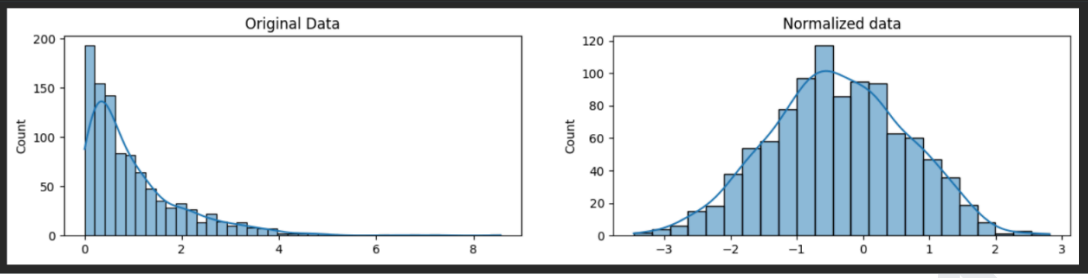
Observation:

The transformed histogram becomes approximately bell-shaped, indicating that the data is more normally distributed.

Purpose:

- Reduce data skewness.
- Improve statistical analysis.
- Prepare data for algorithms that assume normality.

```python
# normalize the exponential data with boxcox
normalized_data = stats.boxcox(original_data)

# plot both together to compare
fig, ax=plt.subplots(1, 2, figsize=(15, 3))
sns.histplot(original_data, ax=ax[0], kde=True, legend=False)
ax[0].set_title("Original Data")
sns.histplot(normalized_data[0], ax=ax[1], kde=True, legend=False)
ax[1].set_title("Normalized data")
plt.show()
```



Step: Import Libraries for Date Parsing

Description:

The required libraries are imported to demonstrate date parsing operations. The NFL dataset is loaded into a DataFrame, and NumPy is used to ensure reproducible results.

Purpose:

- Import data manipulation libraries.
- Load the dataset.
- Prepare data for date formatting operations.

```python
# modules we'll use
import pandas as pd
import numpy as np
import seaborn as sns
import datetime

# read in our data
landslides = pd.read_csv("/content/NFL Play by Play 2009-2016 (v3).csv")

# set seed for reproducibility
np.random.seed(0)
```

```
/tmp/ipykernel_1921/1069185768.py:8: DtypeWarning: Columns (25,51) have mixed types. Specify dtype option on import or set low_memory=False.
  landslides = pd.read_csv("/content/NFL Play by Play 2009-2016 (v3).csv")
```

Step: Display Dataset

Description:

The `head()` function is used to display the first five records of the dataset. This helps identify the available columns, including the **Date** column that will be converted into DateTime format.

Purpose:

- Inspect dataset contents.
- Verify successful loading.
- Identify the date column.

```
landslides.head()
```

	Date	GameID	Drive	qtr	down	time	TimeUnder	TimeSecs	PlayTimeDiff	SideofField	...	yacEPA	Home_WP_pre	Away_WP_pre	Home_WP_post	Away_WP_post	Win_Prob	
0	2009-09-10	2009091000	1	1	NaN	15:00	15	3600.0	0.0	TEN	...	NaN	0.485675	0.514325	0.546433	0.453567	0.485675	0.0
1	2009-09-10	2009091000	1	1	1.0	14:53	15	3593.0	7.0	PIT	...	1.146076	0.546433	0.453567	0.551088	0.448912	0.546433	0.0
2	2009-09-10	2009091000	1	1	2.0	14:16	15	3556.0	37.0	PIT	...	NaN	0.551088	0.448912	0.510793	0.489207	0.551088	-0.0
3	2009-09-10	2009091000	1	1	3.0	13:35	14	3515.0	41.0	PIT	...	-5.031425	0.510793	0.489207	0.461217	0.538783	0.510793	-0.0
4	2009-09-10	2009091000	1	1	4.0	13:27	14	3507.0	8.0	PIT	...	NaN	0.461217	0.538783	0.558929	0.441071	0.461217	0.0

5 rows × 102 columns

Step: Display Date Column

Description:

The **Date** column is displayed separately to examine its current format before parsing. This confirms that the values are stored as text strings.

Purpose:

- Examine date values.
- Verify date format.
- Prepare for conversion into DateTime.

```
# print the first few rows of the date column
print(landslides['Date'].head())

0    2009-09-10
1    2009-09-10
2    2009-09-10
3    2009-09-10
4    2009-09-10
Name: Date, dtype: object
```

Step: Check Date Data Type

Description:

The data type of the **Date** column is checked using the `.dtype` attribute. The result indicates that the column is stored as an object (string).

Observation:

Data type:

object

Purpose:

- Identify the existing data type.
- Confirm that conversion is required.

```
[21]
# check the data type of our date column
landslides['Date'].dtype

dtype('O')
```

Step: Convert Date to DateTime

Description:

The `pd.to_datetime()` function converts the **Date** column from object format into the DateTime format. This enables date-based operations such as extracting days, months, and years.

Purpose:

- Convert strings into DateTime objects.
- Enable time-series analysis.
- Prepare data for feature extraction.

```
[23]
# create a new column, date_parsed, with the parsed dates
landslides['date_parsed'] = pd.to_datetime(landslides['Date'])
```

Step: Display Parsed Dates

Description:

The newly created **date_parsed** column is displayed to verify that the conversion was successful.

Observation:

The parsed column is stored as **datetime64[ns]**, indicating successful conversion.

Purpose:

- Validate date conversion.
- Ensure correct parsing.

192424051

```
[24]
# print the first few rows
landslides['date_parsed'].head()
```

	date_parsed
0	2009-09-10
1	2009-09-10
2	2009-09-10
3	2009-09-10
4	2009-09-10

dtype: datetime64[ns]

Step: Extract Day of the Month

Description:

The `.dt.day` attribute extracts the day component from every parsed date, creating a new feature that can be used for further analysis.

Purpose:

- Extract meaningful date information.
- Prepare data for temporal analysis.

```
[25]
# get the day of the month from the date_parsed column
day_of_month_landslides = landslides['date_parsed'].dt.day
day_of_month_landslides.head()
```

	date_parsed
0	10
1	10
2	10
3	10
4	10

dtype: int32

Step: Visualize Day Distribution

Description:

A histogram is generated to display the frequency of records occurring on each day of the month. This visualization helps identify patterns and variations in the temporal distribution of the data.

Purpose:

- Analyze date distribution.
- Identify trends and patterns.
- Support exploratory data analysis.

```
[26]
0s
# remove na's
day_of_month_landslides = day_of_month_landslides.dropna()

# plot the day of the month
sns.distplot(day_of_month_landslides, kde=False, bins=31)
```

```
/tmp/ipykernel_1921/13696829.py:5: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  sns.distplot(day_of_month_landslides, kde=False, bins=31)
<Axes: xlabel='date_parsed'>
```

Step: Import Character Encoding Libraries

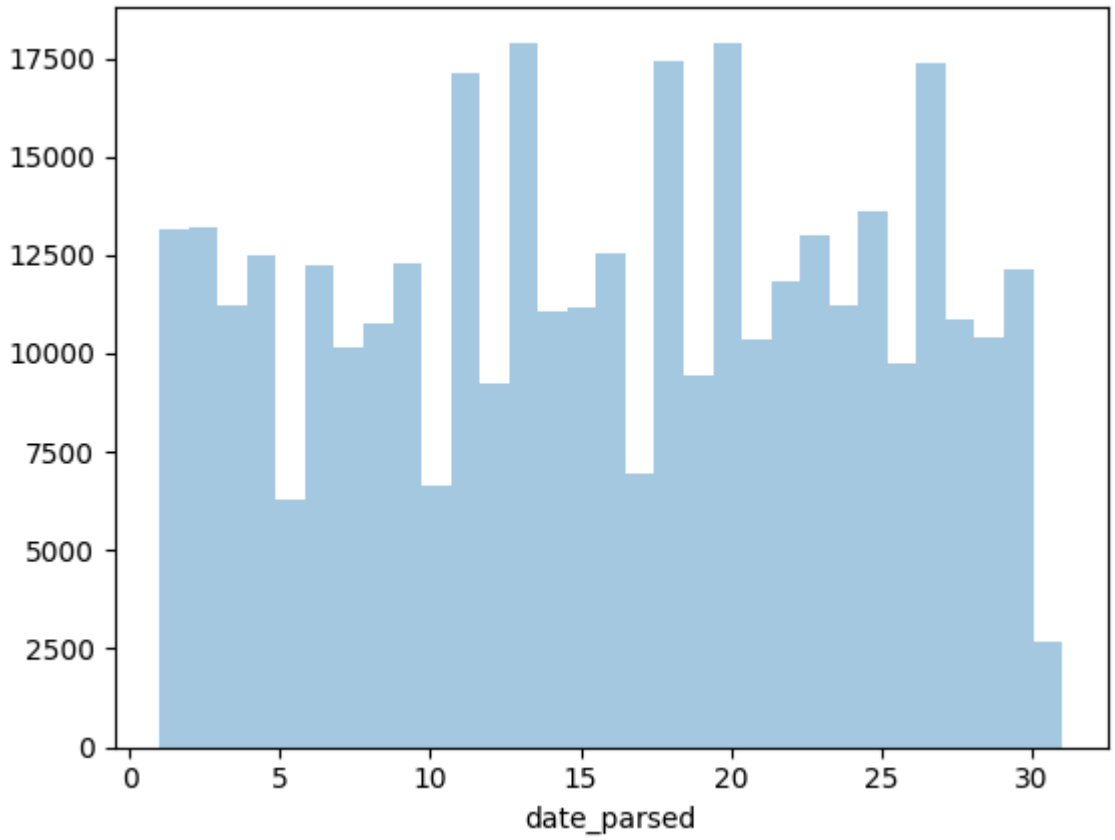
Description:

The required libraries are imported to demonstrate character encoding operations. The `charset_normalizer` package is included to automatically detect the encoding of text files.

Purpose:

- Import encoding libraries.
- Prepare for encoding detection.
- Handle text data correctly.

192424051



Step: Create Sample String

Description:

A sample string containing the Euro (€) symbol is created to demonstrate how Python handles Unicode characters.

Purpose:

- Create sample text.
- Demonstrate Unicode representation.
- Prepare for encoding examples.

```
[28]
✓ 0s
# start with a string
before = "This is the euro symbol: €"

# check to see what datatype it is
type(before)

str
```

Step: Encode Text into UTF-8

Description:

The string is encoded using UTF-8, converting it from a Unicode string into a sequence of bytes.

Purpose:

- Convert text into binary format.
- Demonstrate UTF-8 encoding.
- Prepare for decoding operations.

```
[29]
# encode it to a different encoding, replacing characters that raise errors
after = before.encode("utf-8", errors="replace")

# check the type
type(after)

bytes
```

Step: Display Encoded Bytes

Description:

The encoded byte sequence is displayed to observe how UTF-8 stores special characters internally.

Purpose:

- Understand binary representation.
- Visualize encoded data.

```
[30]
# take a look at what the bytes look like
after

b'This is the euro symbol: \xe2\x82\xac'
```

Step: Decode UTF-8 Text

Description:

The UTF-8 encoded byte sequence is decoded back into readable text, recovering the original string successfully.

Purpose:

- Verify successful decoding.
- Restore readable text.

```
[31]
✓ 0s
# convert it back to utf-8
print(after.decode("utf-8"))

This is the euro symbol: €
```

Step: Demonstrate ASCII Decoding Error

Description:

An attempt is made to decode UTF-8 bytes using ASCII encoding. Since ASCII does not support the Euro (€) symbol, Python raises a `UnicodeDecodeError`.

Purpose:

- Demonstrate encoding incompatibility.
- Understand decoding errors.

```
[32]
0s
# try to decode our bytes with the ascii encoding
print(after.decode("ascii"))

---------------------------------------------------------------------------
UnicodeDecodeError                        Traceback (most recent call last)
/tmp/ipykernel_1921/27547290.py in <cell line: 0>()
      1 # try to decode our bytes with the ascii encoding
----> 2 print(after.decode("ascii"))

UnicodeDecodeError: 'ascii' codec can't decode byte 0xe2 in position 25: ordinal not in range(128)

Next steps: Explain error
```

Step: Replace Unsupported Characters

Description:

The string is encoded using ASCII with the **replace** option, causing unsupported characters to be replaced by a question mark (?).

Observation:

The Euro symbol is replaced with **?**, indicating information loss.

Purpose:

- Handle unsupported characters safely.
- Prevent encoding exceptions.

```
[33]
✓ 0s
# start with a string
before = "This is the euro symbol: €"

# encode it to a different encoding, replacing characters that raise errors
after = before.encode("ascii", errors = "replace")

# convert it back to utf-8
print(after.decode("ascii"))

# We've lost the original underlying byte string! It's been
# replaced with the underlying byte string for the unknown character :(

This is the euro symbol: ?
```

Step: Load Kickstarter Dataset

Description:

The Kickstarter dataset is loaded into a Pandas DataFrame to demonstrate character encoding detection and file handling.

Purpose:

- Load dataset.
- Prepare for encoding analysis.

```
[34]
✓ 1s
# try to read in a file not in UTF-8
kickstarter_2016 = pd.read_csv("/content/ks-projects-201801-utf8.csv")

/tmp/ipykernel_1921/2180336315.py:2: DtypeWarning: Columns (14,15,16) have mixed types. Specify dtype option on import or set low_me
  kickstarter_2016 = pd.read_csv("/content/ks-projects-201801-utf8.csv")
```

Step: Detect File Encoding

Description:

The `charset_normalizer` library analyzes the first few bytes of the dataset to identify its character encoding automatically.

Observation:

The detected encoding is **UTF-8** with a confidence score of approximately **99.1%**.

Purpose:

- Identify file encoding.
- Ensure correct data reading.
- Prevent encoding-related errors during preprocessing.

192424051

```
kickstarter_2016 = pd.read_csv("/content/ks-projects-201801-utf8.csv")
kickstarter_2016.head()
```

```
/tmp/ipykernel_2266/2210720167.py:1: DtypeWarning: Columns (14,15,16) have mixed types. Specify dtype option on import or set low_memory=False.
  kickstarter_2016 = pd.read_csv("/content/ks-projects-201801-utf8.csv")
```

	Unnamed: 0	ID	name	category	main_category	currency	deadline	goal	launched	pledged	state	backers	country	usd pledged	Unnamed: 13	Unnamed: 14	Unnamed: 15	Unnam
0	0	1000002330	The Songs of Adelaide & Abullah	Poetry	Publishing	GBP	2015-10-09 11:36:00	1000	2015-08-11 12:12:28	0	failed	0	GB	0	NaN	NaN	NaN	N
1	1	1000004038	Where is Hank?	Narrative Film	Film & Video	USD	2013-02-26 00:20:50	45000	2013-01-12 00:20:50	220	failed	3	US	220	NaN	NaN	NaN	N
2	2	1000007540	ToshiCapital Rekordz Needs Help to Complete Album	Music	Music	USD	2012-04-16 04:24:11	5000	2012-03-17 03:24:11	1	failed	1	US	1	NaN	NaN	NaN	N
3	3	1000011046	Community Film Project: The Art of Neighborhoo...	Film & Video	Film & Video	USD	2015-08-29 01:00:00	19500	2015-07-04 08:35:03	1283	canceled	14	US	1283	NaN	NaN	NaN	N
4	4	1000014025	Monarch Espresso Bar	Restaurants	Food	USD	2016-04-01 13:38:27	50000	2016-02-26 13:38:27	52375	successful	224	US	52375	NaN	NaN	NaN	N

Reading the Kickstarter Dataset

Description

The Kickstarter dataset is imported into a Pandas DataFrame using the `read_csv()` function. Displaying the first few records verifies that the dataset has been loaded successfully and is ready for further preprocessing and character encoding analysis.

Observation

The dataset is successfully loaded, and the first five rows along with multiple columns are displayed. This confirms that the CSV file has been imported correctly into the Python environment.

Purpose

To load the dataset into Python and verify that it is available for character encoding operations.

```
[70]
✓ 0s
# start with a string
before = "This is the euro symbol: €"

# check to see what datatype it is
type(before)
```

```
str
```

Checking the String Data Type

Description

A sample text string containing the Euro (€) symbol is created. The `type()` function is used to determine the data type of the variable.

Observation

The output shows the data type as `str`, confirming that the text is stored as a Python string.

Purpose

To understand the initial data type of text before encoding it into bytes.

```
# encode it to a different encoding, replacing characters that raise errors
after = before.encode("utf-8", errors="replace")

# check the type
type(after)
```

```
bytes
```

Encoding the String into UTF-8

Description

The string is encoded into UTF-8 using the `encode()` method. UTF-8 converts text into a sequence of bytes suitable for storage and transmission.

Observation

The encoded object is displayed as a byte string, and its data type changes from `str` to `bytes`.

Purpose

To demonstrate how textual data is converted into UTF-8 encoded bytes.

```
[72]
✓ 0s
# take a look at what the bytes look like
after
```

```
b'This is the euro symbol: \xe2\x82\xac'
```

Viewing the Encoded Bytes

Description

The encoded byte sequence is displayed to observe how special characters are represented internally after UTF-8 encoding.

Observation

The Euro symbol is represented by the byte sequence `\xe2\x82\xac`, indicating successful UTF-8 encoding.

Purpose

To visualize the binary representation of Unicode characters in UTF-8 format.

```
[73]
✓ 0s
# convert it back to utf-8
print(after.decode("utf-8"))

This is the euro symbol: €
```

Decoding UTF-8 Bytes

Description

The UTF-8 encoded bytes are decoded back into a readable string using the `decode()` method.

Observation

The original text, including the Euro symbol (€), is successfully restored without any loss of information.

Purpose

To verify that UTF-8 encoding and decoding preserve the original text accurately.

```
[74]
0s
# try to decode our bytes with the ascii encoding
print(after.decode("ascii"))

---------------------------------------------------------------------------
UnicodeDecodeError                        Traceback (most recent call last)
/tmp/ipykernel_2266/27547290.py in <cell line: 0>()
      1 # try to decode our bytes with the ascii encoding
----> 2 print(after.decode("ascii"))

UnicodeDecodeError: 'ascii' codec can't decode byte 0xe2 in position 25: ordinal not in range(128)
```

Decoding UTF-8 Bytes Using ASCII

Description

The encoded byte sequence is decoded using the ASCII encoding instead of UTF-8 to demonstrate encoding incompatibility.

Observation

A `UnicodeDecodeError` occurs because ASCII cannot decode the byte sequence representing the Euro symbol.

Purpose

To illustrate that using an incorrect character encoding can result in decoding errors.

```
[75]
# start with a string
before = "This is the euro symbol: €"

# encode it to a different encoding, replacing characters that raise errors
after = before.encode("ascii", errors = "replace")

# convert it back to utf-8
print(after.decode("ascii"))

# We've lost the original underlying byte string! It's been
# replaced with the underlying byte string for the unknown character :(

This is the euro symbol: ?
```

Replacing Unsupported Characters

Description

The string is encoded using ASCII with the `errors="replace"` option, allowing unsupported characters to be replaced instead of generating an error.

Observation

The Euro symbol is replaced with a question mark (`?`), indicating that unsupported characters are substituted during encoding.

Purpose

To demonstrate how unsupported characters are handled when error replacement is enabled.

```
[78]
# try to read in a file not in UTF-8
kickstarter_2016 = pd.read_csv("../input/kickstarter-projects/ks-projects-201612.csv")

---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last)
/tmp/ipykernel_2266/3982885289.py in <cell line: 0>()
      1 # try to read in a file not in UTF-8
----> 2 kickstarter_2016 = pd.read_csv("../input/kickstarter-projects/ks-projects-201612.csv")

                              4 frames
/usr/local/lib/python3.12/dist-packages/pandas/io/common.py in get_handle(path_or_buf, mode, encoding, compression, memory_map, is_text, errors, storage_options)
    871         if ioargs.encoding and "b" not in ioargs.mode:
    872             # Encoding
--> 873             handle = open(
    874                 handle,
    875                 ioargs.mode,

FileNotFoundError: [Errno 2] No such file or directory: '../input/kickstarter-projects/ks-projects-201612.csv'

Next steps: Explain error
```

Reading a File with Incorrect File Path

Description

The dataset is loaded using an incorrect file path to demonstrate what happens when the specified file cannot be located.

Observation

A `FileNotFoundError` is generated because the CSV file does not exist at the specified location.

Purpose

To understand the importance of providing the correct file path when reading datasets.

```
[79]
# look at the first ten thousand bytes to guess the character encoding
with open("/content/ks-projects-201801-utf8.csv", 'rb') as rawdata:
    result = charset_normalizer.detect(rawdata.read(10000))

# check what the character encoding might be
print(result)

{'encoding': 'utf-8', 'language': 'German', 'confidence': 0.991}
```

Detecting Character Encoding

Description

The `charset_normalizer` library analyzes the first few bytes of the dataset to automatically identify its character encoding.

Observation

The library predicts the encoding (`UTF-8`) along with a high confidence score, indicating reliable detection.

Purpose

To identify the correct encoding before importing datasets containing special characters.

192422051

```
[81]
# read in the file with the encoding detected by charset_normalizer
kickstarter_2016 = pd.read_csv("/content/ks-projects-201801-utf8.csv")

# look at the first few lines
kickstarter_2016.head()
```

```
/tmp/ipykernel_2266/1945819698.py:2: DtypeWarning: Columns (15,16,17) have mixed types. Specify dtype option on import or set low_memory=False.
  kickstarter_2016 = pd.read_csv("/content/ks-projects-201801-utf8.csv")
```

	Unnamed: 0.1	Unnamed: 0	ID	name	category	main_category	currency	deadline	goal	launched	pledged	state	backers	country	usd pledged	Unnamed: 13	Unnamed: 14	Unnam
0	0	0	1000002330	The Songs of Adelaide & Abullah	Poetry	Publishing	GBP	2015-10-09 11:36:00	1000	2015-08-11 12:12:28	0	failed	0	GB	0	NaN	NaN	N
1	1	1	1000004038	Where is Hank?	Narrative Film	Film & Video	USD	2013-02-26 00:20:50	45000	2013-01-12 00:20:50	220	failed	3	US	220	NaN	NaN	N
2	2	2	1000007540	ToshiCapital Rekordz Needs Help to Complete Album	Music	Music	USD	2012-04-16 04:24:11	5000	2012-03-17 03:24:11	1	failed	1	US	1	NaN	NaN	N
3	3	3	1000011046	Community Film Project: The Art of Neighborhoo...	Film & Video	Film & Video	USD	2015-08-29 01:00:00	19500	2015-07-04 08:35:03	1283	canceled	14	US	1283	NaN	NaN	N
4	4	4	1000014025	Monarch Espresso Bar	Restaurants	Food	USD	2016-04-01 13:38:27	50000	2016-02-26 13:38:27	52375	successful	224	US	52375	NaN	NaN	N

Reading the Dataset with the Detected Encoding

Description

The dataset is read using the encoding identified by the character detection library, ensuring correct interpretation of all characters.

Observation

The dataset loads successfully, and the first few rows are displayed without any encoding-related issues.

Purpose

To import the dataset correctly using the detected character encoding.

```
[82]
# save our file (will be saved as UTF-8 by default!)
kickstarter_2016.to_csv("ks-projects-201801-utf8.csv")
```

Saving the Dataset in UTF-8

Description

The cleaned dataset is saved using the `to_csv()` function. By default, Pandas stores CSV files using UTF-8 encoding.

Observation

The dataset is successfully written to a new CSV file in UTF-8 format, making it suitable for use across different platforms.

Purpose

To save the processed dataset in a standardized UTF-8 encoding for future analysis and sharing.

DATA DUPLICATION

```
import pandas as pd

data = {

    'Name': ['Alice', 'Bob', 'Alice', 'Charlie', 'Bob', 'David'],

    'Age': [25, 30, 25, 35, 30, 40],

    'City': ['New York', 'Los Angeles', 'New York', 'Chicago', 'Los Angeles', 'San Francisco']}

df = pd.DataFrame(data)

df
```

	Name	Age	City
0	Alice	25	New York
1	Bob	30	Los Angeles
2	Alice	25	New York
3	Charlie	35	Chicago
4	Bob	30	Los Angeles
5	David	40	San Francisco

Next steps: Generate code with df New interactive sheet

9424051

duplicates = df.duplicated()

duplicates

	0
0	False
1	False
2	True
3	False
4	True
5	False

dtype: bool

df_no_duplicates_columns = df.drop_duplicates(subset=['Name', 'City'])

(df_no_duplicates_columns)

	Name	Age	City
0	Alice	25	New York
1	Bob	30	Los Angeles
3	Charlie	35	Chicago
5	David	40	San Francisco

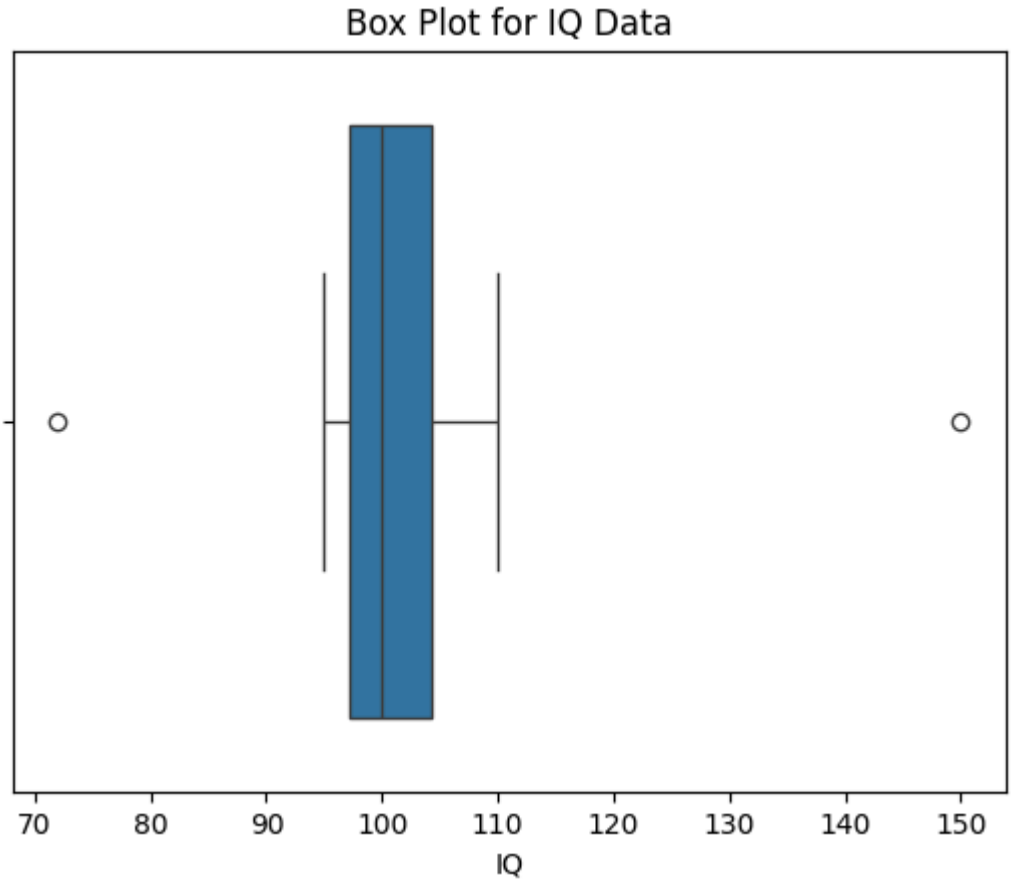
df_keep_last = df.drop_duplicates(keep='last')

(df_keep_last)

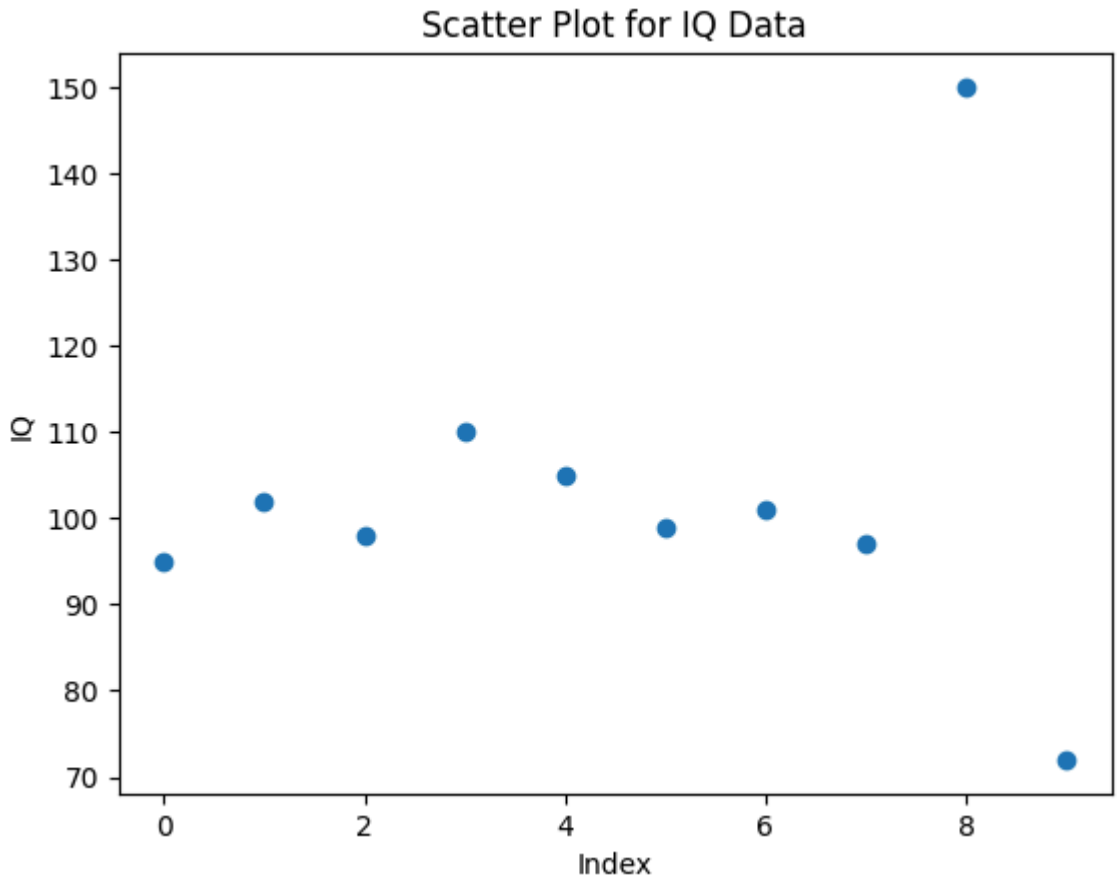
	Name	Age	City
2	Alice	25	New York
3	Charlie	35	Chicago
4	Bob	30	Los Angeles
5	David	40	San Francisco

OUTLIERS

```
import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt

data = {"IQ": [95, 102, 98, 110, 105, 99, 101, 97, 150, 72]}

df = pd.DataFrame(data)

sns.boxplot(x=df["IQ"])

plt.title("Box Plot for IQ Data")

plt.show()
```



```
plt.scatter(df.index, df["IQ"])

plt.xlabel("Index")

plt.ylabel("IQ")

plt.title("Scatter Plot for IQ Data")

plt.show()
```




```
import pandas as pd

import numpy as np

from scipy.stats import zscore

np.random.seed(0)

normal_iq = np.random.normal(100, 5, 100)

outliers = [30, 250]

iq_data = np.concatenate([normal_iq, outliers])
```

```
df = pd.DataFrame({"IQ": iq_data})

df["Z_Score"] = zscore(df["IQ"])

outliers_z = df[np.abs(df["Z_Score"]) > 3]

print(outliers_z)
```

```
...          IQ   Z_Score
100    30.0 -4.156281
101   250.0  8.708296
```

```
Q1 = df["IQ"].quantile(0.25)

Q3 = df["IQ"].quantile(0.75)

IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR

upper_bound = Q3 + 1.5 * IQR

outliers_iqr = df.query("IQ < @lower_bound or IQ > @upper_bound")

print(outliers_iqr)
```

```
...          IQ   Z_Score
100    30.0 -4.156281
101   250.0  8.708296
```

2424051

```
from sklearn.cluster import DBSCAN

dbscan = DBSCAN(eps=5, min_samples=2)

df["Cluster"] = dbscan.fit_predict(df[["IQ"]])

outliers_dbscan = df[df["Cluster"] == -1]

print(outliers_dbscan)
```

```
...           IQ   Z_Score  Cluster
100    30.0 -4.156281       -1
101   250.0  8.708296       -1
```

```
from sklearn.ensemble import IsolationForest

iso = IsolationForest(contamination=0.2, random_state=42)

df["Outlier"] = iso.fit_predict(df[["IQ"]])

outliers_iso = df[df["Outlier"] == -1]

print(outliers_iso)
```

```
...              IQ   Z_Score  Cluster  Outlier
    0    108.820262  0.452761        0       -1
    3    111.204466  0.592178        0       -1
    4    109.337790  0.483024        0       -1
    11   107.271368  0.362189        0       -1
    20    87.235051 -0.809441        0       -1
    24   111.348773  0.600617        0       -1
    25    92.728172 -0.488229        0       -1
    28   107.663896  0.385142        0       -1
    33    90.096018 -0.642145        0       -1
    41    92.899910 -0.478187        0       -1
    42    91.468649 -0.561880        0       -1
    43   109.753877  0.507355        0       -1
    46    93.736023 -0.429295        0       -1
    48    91.930511 -0.534873        0       -1
    63    91.368587 -0.567731        0       -1
    66    91.849008 -0.539639        0       -1
    83    92.318782 -0.512169        0       -1
    85   109.479446  0.491307        0       -1
    97   108.929352  0.459140        0       -1
    100   30.000000 -4.156281       -1       -1
    101  250.000000  8.708296       -1       -1
```

FUZZY DATASET

```
import pandas as pd

from rapidfuzz import process

# Create dataset

data = { "Student_ID": [101, 102, 103, 104, 105, 106, 107, 108], "Name": [ "Rahul", "Priya", "Arun", "Divya", "Meena", "Kavin", "Riya", "Siva" ], "Department": [ "Computer Science", "Compter Science", "Computer Scince", "Information Technology", "Information Tech", "Mechanical", "Mechanicl", "Electronics" ] }

df = pd.DataFrame(data)

print("Original Dataset")

print(df)
```

```
...  Original Dataset
        Student_ID   Name              Department
     0         101  Rahul        Computer Science
     1         102  Priya         Compter Science
     2         103   Arun         Computer Scince
     3         104  Divya  Information Technology
     4         105  Meena        Information Tech
     5         106  Kavin              Mechanical
     6         107   Riya              Mechanicl
     7         108   Siva            Electronics
```

```
import pandas as pd

from rapidfuzz import process

df = pd.DataFrame(data)

print("Original Dataset")

print(df)
```

```
# Standard department names
correct_departments = [
    "Computer Science",
    "Information Technology",
    "Mechanical",
    "Electronics"
]
# Function to find best match
def correct_name(dept):
    match = process.extractOne(dept, correct_departments)
    return match[0]
# Apply fuzzy matching
df["Corrected_Department"] = df["Department"].apply(correct_name)
print("\nDataset After Fuzzy Matching")
print(df)
```

```
Original Dataset
   Student_ID   Name              Department
0         101  Rahul        Computer Science
1         102  Priya         Compter Science
2         103   Arun         Computer Scince
3         104  Divya  Information Technology
4         105  Meena        Information Tech
5         106  Kavin              Mechanical
6         107   Riya               Mechanicl
7         108   Siva             Electronics

Dataset After Fuzzy Matching
   Student_ID   Name              Department    Corrected_Department
0         101  Rahul        Computer Science        Computer Science
1         102  Priya         Compter Science        Computer Science
2         103   Arun         Computer Scince        Computer Science
3         104  Divya  Information Technology  Information Technology
4         105  Meena        Information Tech  Information Technology
5         106  Kavin              Mechanical              Mechanical
6         107   Riya               Mechanicl              Mechanical
7         108   Siva             Electronics             Electronics
```

24051

192424051